

api.dez.ro

Developer Guide

Last updated: 2026-06-30

This guide explains how apps consuming this API are meant to work: the principles behind the design, and the typical sequences of calls ("workflows") for each screen or feature. It is written for app developers building against api.dez.ro and complements the field-by-field API Reference.

Use this guide to understand why and when to call things; use the API Reference to look up the exact request/response shape once you know which endpoint you need.

1. The Big Picture

An app build targets api.dez.ro exactly one medium and sends that value on every request — it never switches at runtime.

X-MEDIUM	Powers	Language
dezro	dez.ro — Romanian car/truck/tire parts	Romanian

Any logged-in user can create and manage ads.

2. Every Request Needs Two Headers

```
Authorization: Bearer <API_KEY>
X-MEDIUM: dezro
```

Both are required on every request, including ones that do not need a logged-in user (catalog browsing, posting a request anonymously). They are checked before any routing or business logic runs:

Problem	Response
Missing/incorrect Authorization	401 {"error":"Unauthorized"}
Missing X-MEDIUM	400 {"error":"Missing required 'X-MEDIUM' header"}
Invalid X-MEDIUM value	400 {"error":"Invalid 'X-MEDIUM' value"}

API_KEY is an app-level secret (one per consuming app/system), not tied to a specific user — it identifies "this is a request from our app", separately from X-Session-Token, which identifies which user.

A minimal request:

```
curl https://api.dez.ro/brands \
-H "Authorization: Bearer <API_KEY>" \
-H "X-MEDIUM: dezro"
```

3. Sessions: Log In Once, Reuse the Token

```
curl -X POST https://api.dez.ro/user/login \
-H "Authorization: Bearer <API_KEY>" \
-H "X-MEDIUM: dezro" \
```

```
-H "Content-Type: application/json" \
-d '{"identifier": "johndoe", "password": "secret123"}'

{ "data": { "token": "789cbfa7..." } }
```

What to do with token:

- Store it (secure storage / keychain) and send it as X-Session-Token on every subsequent request that needs a logged-in user.
- It is valid for 30 days from issue — not a sliding window, so it expires even if the app is used daily. When a request comes back 401 {"error": "Invalid or expired session token"}, drop the stored token and send the user back to the login screen.
- Each login issues a new, independent token. Logging in on a second device does not invalidate the first device's token, and POST /user/logout only invalidates the one token it is called with.
- **Do not call /user/login on every app launch** — reuse the stored token until it is rejected or the user explicitly logs out.
- Failed logins are rate-limited per identifier+IP (5 attempts / 15 minutes) — 429 with a Retry-After header. Surface this as "too many attempts, try again in N seconds" rather than retrying automatically.

Once you have a token, fetch the profile:

```
curl https://api.dez.ro/user \
-H "Authorization: Bearer <API_KEY>" \
-H "X-MEDIUM: dezro" \
-H "X-Session-Token: 789cbfa7..."
```

4. Reading Responses & Errors

Every response is JSON with one of two shapes:

```
{ "data": { ... } } // success
{ "error": "Human readable" } // failure
```

- Use the HTTP status code to decide success/failure (2xx vs 4xx/5xx), then read data/error accordingly — do not infer success from the body alone.
- error messages (and message fields in success responses) are already translated based on X-MEDIUM (dezro → Romanian). Display them to the user as-is; do not re-translate client-side.
- The one exception: errors that happen before X-MEDIUM is validated (bad Authorization, missing/invalid X-MEDIUM) are always in English — these indicate an app bug, not something to show the user.

A practical error-handling checklist:

Status	Typical meaning	What to do
401 (Missing / Invalid or expired session token)	Session expired or not logged in	Clear the stored token, show the login screen
403 (Orders access disabled)	User is not a "yard" (anymore)	Hide/disable yard-only screens, do not treat as a hard error
422	Bad input (validation)	Show the error message near the relevant field — it is already user-facing text
404	Resource not found / not yours	Treat like "no longer exists" (e.g. order deleted from another device)
429	Rate limited (login only)	Show a "try again later" message using Retry-After

5. Building the Catalog Pickers (Brands, Parts, Geo)

Before a user can post an ad, your "new request" form needs brand/model and part category. These come from GET /brands and GET /parts— all available without a session token, and all cached server-side for a day, so call them freely.

Everything at once

For a form that is all on one screen — fetch the full trees in one response each. Good for caching client-side at app startup:

```
curl "https://api.dez.ro/brands?with_models" \
-H "Authorization: Bearer <API_KEY>" -H "X-MEDIUM: dezro"

curl "https://api.dez.ro/parts?with_children" \
-H "Authorization: Bearer <API_KEY>" -H "X-MEDIUM: dezro"
```

Drill-down

For a multi-step picker (pick a brand, then load its models):

```
curl "https://api.dez.ro/brands/car" \
-H "Authorization: Bearer <API_KEY>" -H "X-MEDIUM: dezro"

curl "https://api.dez.ro/brands/car/1/models" \
-H "Authorization: Bearer <API_KEY>" -H "X-MEDIUM: dezro"
```

For parts categories, check selectable:

selectable value	Meaning	UI behaviour
0	Grouping category only (e.g. "Engine")	Cannot be chosen — must drill into its children
1	Leaf / selectable category	Can be chosen directly

tire brands (dezro only) have no models — do not request with_models or /models for them.

6. Managing Classified Ads

Ads are seller-driven listings — a parts yard or private seller posts what they have in stock. A logged-in user can create, update, and delete their own ads; there is no separate "seller account" gate.

Ads created via the API are not published immediately and they do not appear live on the site — there is an admin approval step for API-submitted ads. Once approved, the URL field in the response contains the full public URL on dez.ro that you can show as a "view your live ad" link.

6.1 Load the Available Ad Types

Ad types differ per medium. Fetch them once at session start and cache them client-side — they are static reference data. No session token is needed.

```
curl https://api.dez.ro/ads/types \
-H "Authorization: Bearer <API_KEY>" \
-H "X-MEDIUM: dezro"

{
  "data": {
    "types": [
      { "id": 0, "label": "Piese Auto" },
      { "id": 1, "label": "Dezmembrari" },
      { "id": 2, "label": "Masini Avariate" },
      { "id": 3, "label": "Anvelope SH" },
    ]
  }
}
```

```
{ "id": 4, "label": "Piese camioane" }
]
```

Use this endpoint to populate the type picker in your "create ad" form; do not hardcode the labels, since they are medium-specific and may change.

6.2 Create an Ad

```
curl -X POST https://api.dez.ro/ads \
-H "Authorization: Bearer <API_KEY>" \
-H "X-MEDIUM: dezro" \
-H "X-Session-Token: 789cbfa7..." \
-F "title=Janta aliaj BMW Seria 3 2019" \
-F "type=0" \
-F "idBrand=1" \
-F "idModel=10" \
-F "idPart=42" \
-F "year=2019" \
-F "price=350" \
-F "qty=4" \
-F "description=Jante originale, fara lovituri" \
-F "variant=F30" \
-F "oem=36116796207" \
-F "images[]=@wheel1.jpg" \
-F "images[]=@wheel2.jpg"
```

Required fields: title, description, type, idBrand. idModel is required for all types except 3 (tires). idPart is required only for types 0 (parts) and 4 (truck parts). Everything else is optional.

Field	Notes
type	Integer from GET /ads/types
idBrand / idModel	From GET /brands — same catalog used for orders
idPart	From GET /parts
year	4-digit year, or 0 / omit for "year unknown"
price	Integer in local currency (RON for dezro); 0 = not specified
qty	Quantity in stock; 0 = not specified
variant	Free text — body style, engine variant, etc.
oem	OEM / manufacturer part number
images[]	JPEG, PNG, or WebP; max 8 MB each; up to 10 images total

Location (county / locality) is resolved automatically from the user's profile — do not include it in the request body.

Response 201:

```
{
  "data": {
    "ad": {
      "id": 123,
      "alias": "janta-aliaj-bmw-seria-3-2019",
      "approved": true,
      "url": "https://www.dez.ro/janta-aliaj-bmw-seria-3-2019-123.html",
      "images": [
        { "id": 11, "url": "https://www.dez.ro/media/images/ads/5/123/janta-aliaj-bmw-seria-3-2019.jpg" },
        { "id": 12, "url": "https://www.dez.ro/media/images/ads/5/123/janta-aliaj-bmw-seria-3-2019-1.jpg" }
      ]
    }
  }
}
```

approved is always true for API-created ads. url is the live public URL — display it immediately as a "view your listing" link. Images are watermarked by the legacy site before storage — the URLs in the response are the final, displayable versions.

6.3 List My Ads / Get Counts

Paginated list:

```
curl "https://api.dez.ro/ads?page=1" \
-H "Authorization: Bearer <API_KEY>" \
-H "X-MEDIUM: dezro" \
-H "X-Session-Token: 789cbfa7..."
```

Returns all non-deleted ads owned by the logged-in user, newest first (10 per page). Each item includes the full ad shape (same as GET /ads/{idAd}), so the list screen can render all fields without a second request.

Counts summary — useful for a badge or a "pending / live / total" indicator:

```
curl "https://api.dez.ro/ads?count" \
-H "Authorization: Bearer <API_KEY>" \
-H "X-MEDIUM: dezro" \
-H "X-Session-Token: 789cbfa7..."

{ "data": { "counts": { "total": 5, "approved": 5, "pending": 0 } } }
```

For API-created ads pending should always be 0. approved = live on the site.

6.4 View a Single Ad

```
curl https://api.dez.ro/ads/123 \
-H "Authorization: Bearer <API_KEY>" \
-H "X-MEDIUM: dezro" \
-H "X-Session-Token: 789cbfa7..."
```

Returns the full ad shape. url is a live link to the listing page on dez.ro. The url is stable — the slug never changes on edit, even if the title does.

6.5 Edit an Ad

```
curl -X POST https://api.dez.ro/ads/123 \
-H "Authorization: Bearer <API_KEY>" \
-H "X-MEDIUM: dezro" \
-H "X-Session-Token: 789cbfa7..." \
-F "price=300" \
-F "description=Jante originale, fara lovitur. Pret negociabil." \
-F "images[]=@wheel3.jpg"
```

Every field is independent and optional — only the fields you send are changed; everything else stays as-is. Images in images[] are appended to the existing set (not replaced), up to a total of 10. To remove an existing image, use §6.6.

The response is the full current state of the ad after the update, including any newly added images. Sending no fields and no images is a harmless no-op that returns the current state — useful as a lightweight refresh.

6.6 Delete a Single Image

Each image in the images[] array carries its own id. Use it to delete that specific image without touching the others:

```
curl -X DELETE https://api.dez.ro/ads/123/image/11 \
-H "Authorization: Bearer <API_KEY>" \
-H "X-MEDIUM: dezro" \
-H "X-Session-Token: 789cbfa7..."

{ "data": { "message": "Image deleted" } }
```

The file is removed from the legacy site immediately. To replace an image, delete the old one and then append the new one via POST /ads/{idAd}.

6.7 Delete an Ad

```
curl -X DELETE https://api.dez.ro/ads/123 \
-H "Authorization: Bearer <API_KEY>" \
-H "X-MEDIUM: dezro" \
-H "X-Session-Token: 789cbfa7..."

{ "data": { "message": "Ad deleted" } }
```

The ad is soft-deleted — it disappears immediately from GET /ads and from the site, but remains in the database for admin review. Confirm with the user before calling this, as there is no undo from the app side.

6. Quick Reference: "I Want To..."

Goal	Endpoint(s)
Log in / out	POST /user/login, POST /user/logout
Populate brand/model/part/county pickers	GET /brands, GET /parts, GET /geo (+ {type}/models/children/localities variants)
Post a new classified ad (live immediately)	POST /ads
Show "My Ads" / counts summary	GET /ads, GET /ads?count
View a single ad	GET /ads/{idAd}
Edit an ad / append images	POST /ads/{idAd}
Delete one image from an ad	DELETE /ads/{idAd}/image/{idImage}
Delete an ad	DELETE /ads/{idAd}